USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

Display Driver

UM-NATOS-015

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-015	1.4 · 2026-08-15	**Complete, verified on hardware** — §5.8 added; the framebuffer claim in §5.7 is overturned	Internal

1. Abstract

nat-os drives the CYD's ILI9341 panel: 240×320, 16-bit colour, filled regions and text, driven from a native task showing live kernel state.

There is **no framebuffer anywhere in the system**. UM-NATOS-010 §7.2 argued that one should not exist — the panel holds the image in its own GRAM, so a host copy is a redundant 153,600 B against a 158,000 B heap. This report is that argument implemented: every pixel is rendered through a **480-byte** span buffer.

Confirmed on the physical panel: header bar, live numeric fields, an advancing marker, and a strip of colour primaries.

§5 (revision 1.1) replaces the bit-banged transport with the SPI2 peripheral, taking a full-screen fill from 387 ms to 78 ms. The staging that §3 argues for is what made that change safe to attempt, and §5.3 records why the speedup is 5× rather than the 12× this report originally predicted.

2. Pin map

Taken from the vendor project's active `TFT_eSPI/User_Setup.h` rather than from recollection. On a board where a wrong pin and a wrong init sequence produce an identical symptom, the difference between reading and remembering is the difference between a first-try success and an afternoon.

Signal	GPIO	Note
SCLK	14	
MOSI	13	
MISO	12	unused — the driver never reads the panel
CS	15	
DC	2	low = command, high = data
BL	21	active high
RST	—	**not wired to a GPIO**; follows the ESP32's own reset

Because RST is not under software control, the only reset available after boot is the panel's `0x01 SWRESET`.

3. Why bit-banged SPI

Deliberate, and deliberately temporary.

Hardware SPI2 requires DPORT peripheral clock-enable bits and GPIO matrix signal routing. Every mistake in either produces **exactly the same symptom** as a wiring error, a wrong pin, or a bad init sequence: a black screen with no diagnostic. Bringing up four uncertain subsystems at once and then bisecting them from a single bit of output is how M2 consumed nine build cycles (UM-NATOS-009 §6).

Bit-banging touches only GPIO registers, which `gpio.h` covers completely. A failure could therefore only be the panel, the pins, or the commands — three candidates instead of seven.

3.1 Measured cost

```
153,600 bytes in 387 ms  ≈ 397 kB/s  ≈ 3.2 MHz effective clock
```

The estimate written into the source first was ~150 ms, wrong by a factor of 2.6. A store to a peripheral register costs considerably more than the "few cycles" assumed. The figure is now measured at init and reported over UART, because throughput is the number that decides whether SPI2 is worth the risk, and an assertion would have decided it wrongly.

2.6 full screens per second is ample for a status display and useless for animation. Partial updates are what make it comfortable: redrawing six numeric fields costs a few hundred spans rather than 76,800 pixels.

4. Rendering without a framebuffer

`display_fill_rect()` composes one row into `g_line[240]` — 480 bytes — sets the panel's address window once, then pushes that span `h` times. `display_text()` does the same per glyph row.

The panel's GRAM is the framebuffer. Setting a window and streaming into it is not a workaround for having too little RAM; it is how the device is designed to be driven, and the 153,600 B figure in UM-NATOS-010 §7 was only ever the cost of declining to use it.

4.1 The font is free

A 5×8 column-encoded font, 95 glyphs, 475 bytes, in `.rodata`. Since UM-NATOS-011 that is mapped from flash through the data cache and costs **zero DRAM**. This is the first consumer of that work, and the reason it was scheduled before M4 rather than after.

4.2 Guarded by a mutex

The span buffer and the panel's current address window are both shared state. Two tasks drawing concurrently would interleave pixel streams into a single window and produce garbage.

`display.c` takes a recursive mutex around `fill_rect` and `text` — the first use of UM-NATOS-014's primitive outside its own self-test. Recursive matters here: `display_clear()` draws through `display_fill_rect()`, so a non-recursive lock would deadlock on the first clear.

5. Hardware SPI2

Added in revision 1.1.

5.1 The deferral was the point

§3 argues for bit-banging first because a wrong DPORT clock bit or a wrong IOMUX selection produces a black screen — the same symptom as a wrong pin, a bad init sequence, or an unseated flex. Bringing up four uncertain subsystems together and bisecting them from one bit of output is how M2 consumed nine build cycles.

That argument is what made this change cheap when it came. Panel, pins, command sequence and colour order were already known good, so the only thing under test was the transport. A black screen could mean exactly one thing.

5.2 What it took

The CYD's display pins are the ESP32's **native HSPI pads**, so IOMUX routes SCLK and MOSI straight to the peripheral — no GPIO matrix, and none of the 40 MHz ceiling the matrix imposes.

CS and DC stay ordinary GPIOs. The driver holds CS asserted across a window command *and* its entire pixel stream (§4), which is not a shape the peripheral's CS automation expresses.

Two details worth keeping:

- **The DPORT write is read-modify-write, never a plain store.** Bit 1 of the same register clocks the flash controller this kernel executes from.
- **Both configuration registers are read back.** `clk=0x00001001` confirms the `sysclk/2` divisor took; `dport` bit 6 confirms the peripheral clock gate stuck. A register that silently fails to take is the difference between a fast display and a black one, and this report's whole staging argument is about not having to guess which.

The bit-banged path is retained behind `DISPLAY_USE_SPI2 0`, and both backends now route through a single `spi_tx()` entry point so a switch cannot leave a stray direct-transport call behind.

5.3 Measured, and why the estimate was wrong

```
387 ms  ->  78 ms      full-screen fill
```

5×, against the ~12× this report predicted in revision 1.0.

At 40 MHz the theoretical floor for 153,600 bytes is about 31 ms, so roughly 2.5× of what remains is per-transaction overhead. The FIFO holds 64 bytes, so a 480-byte span costs eight transactions — sixteen register writes, a start and a poll each — and a full screen is **2,560 of them**.

The original estimate treated the clock rate as the only variable. It is the transaction count that dominates, and that is precisely what DMA exists to remove. The gap between the estimate and the measurement is more useful than either number alone, which is why both are recorded.

5.4 Clock choice

`sysclk/2` is deliberate and conservative. 80 MHz is one register bit away (`SPI_CLK_EQU_SYSCLK`), but the practical limit here is the flex cable and the board layout rather than the controller, so raising it is a change to make against a measurement rather than on spec-sheet optimism.

5.5 DMA

Revision 1.2. The FIFO path costs 2,560 transactions per full screen (§5.3). DMA takes a whole 480-byte span in one, so a screen becomes 320 transfers.

```
387 ms   bit-banged GPIO
 78 ms   SPI2, CPU-driven FIFO
 43 ms   SPI2 + DMA          dma=320/0
~31 ms   theoretical floor at 40 MHz
```

320 transfers and zero timeouts is exactly one per row, which confirms every span went through a descriptor rather than falling back.

Written so that failure is diagnosable rather than fatal. A DMA engine that never asserts completion would hang the display task forever and take the system with it — the failure mode this project has already paid for twice, in the `task_yield` freeze (UM-NATOS-016 §3) and the zero-overhead `LOOP` defect (UM-NATOS-009 §6), both of which presented as a stopped kernel. So:

- every wait is bounded by `CCOUNT` and counted, never a bare `while`
- a timeout **disables DMA permanently** and falls through to the FIFO path. An engine that missed one completion has no claim on the next, and a degraded display beats a stopped one
- the configuration registers are read back and reported beside the counters

Descriptors are written as explicit words rather than C bitfields, whose bit order is implementation-defined while this layout belongs to the silicon. The descriptor and the transmit buffer are both in `.bss`, which the linker places in DRAM — a buffer in IRAM would be silently unreachable by the DMA engine.

Transfers under about a FIFO's worth stay on the CPU path, where descriptor setup would cost more than it saves. Commands and their parameter bytes are all in that range.

5.6 Two defects a renderer exposed

Revision 1.3. Driving the display from a raycaster rather than a status screen put a very different load on this layer and found two things.

`display_blit` drew row by row. A 1-pixel-wide column therefore became 168 separate two-byte transfers. When the source is contiguous the whole rectangle is one linear stream, because the panel walks the address window itself — the blitter now detects `stride == width` and streams it. The row-by-row path remains for genuinely strided sources.

Every drawing call locks, and that is about frequency, not correctness. A frame taking the display mutex once per column acquired it 240 times while the applications were also drawing, and each contended acquisition costs a full scheduling round-trip:

```
blit time   25,075,460 us  ->  129,000 us      194x
```

That is UM-NATOS-014 §5.2 exactly — *a blocking mutex is the wrong instrument for a lock contended at high frequency* — rediscovered by walking into it after writing it down. `display_lock()` and `display_unlock()` now let a caller hold across a batch.

5.7 A framebuffer that does not pay

A framebuffer for the view region was added on the reasoning that 120 address windows per frame must dominate. It is switchable at runtime, and the switch is what settled it:

```
fb off:  blit 126,650 us
fb on:   blit 131,411 us
```

No measurable difference. 80,640 B for nothing, so it is implemented, kept, and defaulted **off**.

The reasoning behind it was wrong twice over. `xt_ccount()` deltas taken across preemptible code measure **wall clock**, including every moment the task spent descheduled — so the "450 us per window" that motivated the framebuffer was mostly other tasks running. Killing the applications moved the frame rate from 8 per 6 s to 19, a 2.4× change from freeing CPU alone, which located the real limit: the frame is about 37 ms of work and the display task's share of the CPU decides everything else.

Keeping the switch means the claim can be rechecked whenever the display path changes underneath it, rather than being inherited as folklore.

> **Superseded — see §5.8.** The switch was rechecked and the conclusion did not survive. Both figures above were taken on a clock that intermittently stalled (UM-NATOS-008 §8) and under draw-lock contention that dominated the frame (UM-NATOS-014 §10). With both fixed the framebuffer is worth having and is now the default. The paragraph above is left standing because the *method* — keep the switch, recheck the claim — is what produced the correction.

5.8 The renderer since, and a claim that did not survive

Added in revision 1.4.

The framebuffer pays after all

§5.7 measured no difference between the framebuffer and per-column blitting, and defaulted it off. Both measurements were taken through two faults discovered later:

- `xt_ccount()` deltas measured wall clock, and the tick those frames were counted against **stalled for up to 183 ms at a time** (UM-NATOS-008 §8)
- the draw lock was contended hard enough that the renderer spent most of every frame blocked rather than drawing (UM-NATOS-014 §10)

With both fixed, one window per frame beats 240, and the framebuffer is on by default. §5.7's *conclusion* was wrong; its *method* was right, and is why the error was findable — a switch that can be flipped is a claim that can be rechecked.

Where the frame goes now

```
march      2.6 ms      one ray per screen column
compose   13.9 ms      240 x 168 pixels into DRAM
blit      41.9 ms      one 80,640-byte window
                       ~9.1 fps
```

The frame is bus-bound: 72% of it is pushing pixels down SPI at 40 MHz, which UM-NATOS-018 §4 established is this board's ceiling — 80 MHz works electrically and puts visible noise on the glass. So detail is cheap here and pixels are expensive, and that shapes what is worth adding.

One ray per column

`RAY_COLW` 2 → 1, doubling horizontal detail. The estimate for this was wrong by a factor of forty — predicted 0.2 ms, cost about 9 ms — for two reasons worth recording:

- the 0.2 ms march figure was sampled with the camera **against a wall**, where rays terminate immediately. A cost measured at the cheapest moment is not a cost.
- compose doubled although the pixel count did not: the per-row loop now runs once per column, 240 × 168 iterations instead of 120 × 168 writing two pixels each. Same pixels, twice the loop.

Face shading

Every wall face was equally bright, so two walls meeting at a corner differed only by cell hue and the corner read as a colour change rather than as geometry. Faces crossed through a vertical boundary now render at full brightness and the others at 5/8.

The marcher is fixed-step rather than DDA, so it does not know which face it crossed. It is recovered by comparing the cell one step back — one subtraction, cheaper than replacing the marcher. When both coordinates change cell in the same step the hit is a corner, either answer is defensible, and X wins arbitrarily.

Cost: one comparison per hit and one multiply per column, unmeasurable against a 41.9 ms blit.

Wall texture

Four panels per cell with a narrow dark seam between them, positioned in **world space** so perspective compresses them as a wall recedes. A flat colour gives the eye no scale; evenly spaced marks that converge do. The same trick as the face shading, one level finer.

The seam position comes from where along the face the ray landed — the fraction of whichever coordinate did *not* change cell at the hit. Computed once per column, so it costs a shift and a compare.

Vertical only, and that is a cost decision rather than an aesthetic one: horizontal courses would need per-pixel work inside the compose loop, which is the one place in this renderer where a cost multiplies by 168.

Navigation belongs to the camera, not the driver

The camera's wander was rewritten from reactive probing to a heading chosen once per cell, because no amount of probe tuning could traverse a maze — in corridors one cell wide a look-ahead probe is blocked almost always, so the camera turned continuously and never committed to a direction. Distinct cells visited in 20 s went from 4, pacing in one pocket, to 12 across the map.

That is recorded here for want of a better home. It is not a display-driver property, and if the renderer grows much further it wants its own report rather than a subsection of the driver's.

5.9 The DMA stall, and three failed fixes

Status: unresolved and deliberately left alone. The driver ships with the behaviour described in §5 and the 3D view works. This section is the evidence, so the next attempt starts from it rather than from a fresh guess.

5.9.1 What is happening

DMA disables itself within about eight seconds of the raycaster starting. After that every transfer takes the 64-byte FIFO path — 2,400 transactions per full screen instead of 320 — and the blit costs 55.9 ms instead of a possible ~22 ms. **The 3D view has run this way for its entire existence.** Nothing reported it; the fallback is silent by design.

5.9.2 The stall is spurious

Captured at the moment the guard fires, with the guard left intact:

```
stall after 63910 good transfers, asking 480 B
cmd        = 0x00000000        <- USR bit already CLEAR
dma_status = 0x00000000
after that : FINISHED late after 0 us
waited     = 30332 us
```

The transfer had completed. `cmd` reads zero, and the follow-up poll found it done in 0 µs.

The mechanism is a race between two adjacent lines:

```
while (GPIO_REG(SPI2_CMD) & SPI_USR_BIT) {      /* A: still running? */
    if ((xt_ccount() - start) > 2000000u) {     /* B: too long?      */
```

A preemption landing between A and B times out a transfer that has already finished. `xt_ccount()` is wall-clock, so descheduled time counts against the bound. Rare per transfer — 63,910 succeeded first — and certain over the ~2,900 waits a second the raycaster issues.

5.9.3 The corruption is the recovery, not the timeout

`spi2_dma_tx()` returns 0 on timeout, and `spi_tx()` then falls through to `spi2_tx()` — **re-sending bytes the DMA has already sent.** The panel takes the span twice, its window advances one span too far, and every row after it shifts.

This is a latent bug in the shipped driver, not something introduced by the attempts below. It fires once per boot and is invisible, because the same timeout that causes it also disables DMA so it cannot happen again.

5.9.4 What was tried, and what it cost

attempt	result
Overlap conversion with DMA (two buffers)	no measured gain; broke the view once DMA actually ran
Bound by CPU time, reset and retry instead of disabling	blit 21.9 ms — and the view rendered wrong
Bound by CPU time, re-read USR, never re-send	view still wrong

Three attempts, three breakages. Each had a plausible mechanism and a good number **before anyone looked at the screen**, which is the whole lesson: this driver's only correctness instrument is a person looking at the panel, exactly as §5.3 already says about the SPI clock ceiling.

The pipelined path deserves its own note. It measured as noise (55.9 → 55.5 ms) and was kept anyway as "correct and free". It had also **never actually run** — DMA disabled itself before it mattered — so it sat in the tree for a day looking like tested code. An optimisation with no measured gain has no claim on the tree, and code that only executes when another bug is absent has been tested by nothing.

5.9.5 If this is picked up again

- The 55.9 ms blit is correct. It is slow, and it works. That trade has already been made three times in the other direction and lost every time.
- Fix the duplicate re-send first (§5.3.3). It is a real defect, it is independent of the timing question, and it makes any later timeout change safe rather than destructive.
- Do not remove the guard to study the failure. That was the first mistake.
- Verify on the glass before reporting a number.

6. Verification

Visual confirmation on the panel: the status screen renders, values update, and the colour strip shows distinct primaries.

The colour strip exists as a diagnostic rather than decoration. Eight primaries in a known order make a pixel-format or byte-order fault immediately visible; a garbled or monochrome strip would have said which of the two was wrong, without inference from a photograph.

Colour order confirmed. Red renders leftmost, matching the intended `RED, GREEN, BLUE, YELLOW, CYAN, MAGENTA, WHITE, GREY`. That validates the BGR bit in `MADCTL 0x48`: had the panel's red and blue channels been transposed, the strip would have read blue-first and every colour in the system would have been mirrored while still looking entirely plausible in isolation. Distinct primaries alone would not have caught it — only their **order** does, which is why the strip is drawn in a known sequence rather than as arbitrary swatches.

The advancing marker block is the same idea applied to liveness: a display whose numbers all look plausible but never change is otherwise indistinguishable from a working one.

```
display : init ok, bytes=153634 fullscreen=387 ms
tasks   : report=0 a=1 b=2 vm=3 apps=4 shell=5 disp=6
```

`153,634` is 240×320×2 for the initial clear plus 34 command bytes — the exact figure a correct full-screen fill produces, which is what made it usable as a check before anything was known to be visible.

Regression. Eight native tasks now scheduling. M3, M4, M5 and locking self-tests all still passing.

7. Metrics

Quantity	Value
Resolution	240×320, RGB565
Framebuffer	none
Span buffer	480 B
Font	475 B, in flash
Full-screen fill	**43 ms** via SPI2 + DMA (78 ms FIFO, 387 ms bit-banged)
SPI clock	40 MHz (`sysclk/2`); ~3.2 MHz bit-banged
Transfers per full screen	320 with DMA (2,560 on the 64-byte FIFO)
Theoretical floor at 40 MHz	~31 ms
Bytes for init + clear	153,634
Native tasks	8
Image size	18,896 B
Rays per frame	240, one per column
Frame cost, march / compose / blit	2.6 / 13.9 / 41.9 ms
Cost of face shading and wall seams	unmeasurable — 9.1 fps before and after
Share of frame spent on the bus	72%
Framebuffer	on by default — §5.7's conclusion overturned in §5.8
SPI clock ceiling on this board	40 MHz (80 MHz is electrically fine and visibly noisy)

8. What this does not establish

- **No descriptor chaining.** Each span is a single descriptor started and waited on individually. Chaining whole frames would remove the remaining per-span synchronisation, which is most of the gap to the 31 ms floor.
- **No 80 MHz.** One register bit, but unmeasured against this board's flex and layout (§5.4).
- **No read phase.** The driver is write-only; `MISO` is wired but unused, so the panel is never interrogated.
- **No touch.** The CYD's XPT2046 is on separate pins and entirely untouched.
- **No orientation control.** MADCTL is fixed at `0x48` . Rotation is not implemented.
- **No clipping of text.** A string running past the right edge stops at the panel boundary rather than wrapping.
- **No damage tracking.** Callers decide what to redraw; nothing computes dirty regions. §5.7 makes this the interesting gap: the frame cost is dominated by redrawing everything every frame, not by the transport.

- **No display access from applications.** There is no syscall for drawing, so bytecode cannot reach the screen. That is the obvious next step and would be the first VM syscall to carry a pointer, which needs the same bounds discipline as everything else in UM-NATOS-012 §3.
- **No gamma correction.** The gamma tables (`0xE0` / `0xE1`) are left at panel defaults, so colours are correct in channel but not calibrated in response.
- **Wall texture is vertical only.** Panel seams run floor to ceiling; there are no horizontal courses, because those need per-pixel work where this renderer's costs multiply by 168.
- **Face shading is by orientation only.** There is no light source and no falloff across a face; two walls of the same orientation are equally bright regardless of where they are. It is a depth cue, not lighting.
- **The corner case in face recovery is arbitrary.** When a march step crosses both boundaries at once, X is chosen because something must be. Nothing measures how often that happens or whether it is visible.
- **Nothing measures whether the icons or the shading are legible.** Both were judged by one person on one panel, which is the same standard the report set criticises elsewhere.

9. References

- UM-NATOS-010 §7.2 — why no framebuffer, and the 153,600 B it would have cost
- UM-NATOS-011 — flash-mapped `.rodata` , which the font depends on
- UM-NATOS-014 — the mutex guarding the span buffer
- UM-NATOS-009 §6 — the multi-variable bring-up this driver's staging avoids
- `kernel/gpio.h` — the two layers that must agree before a pin drives anything
- `kernel/display.c` — SPI, init sequence, span rendering, font